



Challenges of Microservices

Understanding the Real-World Trade-offs



Agenda

1. Introduction and Complexity
2. Technical Challenges
3. Coordination, Security, and Summary



Introduction and Complexity



Why Microservices Aren't a Silver Bullet

Flexibility Comes with a Price

- Microservices enable independent scaling and deployment.
- Increased architectural and operational complexity.

Not a Universal Solution

- Best suited for large, complex projects —not every team or app.

Key Challenges Await

- Prepare for distributed systems, data consistency, and coordination hurdles.



The Complexity Trade-off

Monoliths: Simpler Operations

Centralized deployment and monitoring reduce operational burden.

Microservices: Simpler Code, Complex Operations

Individual services are easier to understand, but managing them is harder.

Complexity Shifts to Infrastructure

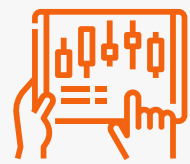
Requires advanced DevOps, automation, and orchestration tools.



Technical Challenges



Distributed System Headaches



Network Failures Are Inevitable

Services may be unreachable or respond slowly at any time.



Resilience Patterns Are Essential

Implement retries, timeouts, and circuit breakers to handle failures.



Eventual Consistency Replaces ACID

No global transactions; data may be temporarily inconsistent.



Data Consistency Gets Tricky

No Global Transactions

Each microservice manages its own database, eliminating cross-service ACID transactions.

Risk of Inconsistent Data

Failures in one service can leave data out of sync across the system.

Requires Advanced Patterns

Sagas and compensation logic are needed to restore consistency after failures.





Operational Overhead Increases

Managing a Proliferation of Services

Each microservice requires its own deployment pipeline and server resources.

DevOps and Automation Become Critical

Automation and strong DevOps skills are essential to handle increased operational demands.

Higher Upfront Infrastructure Costs

Initial investment in tooling, monitoring, and orchestration is significant.



Testing and Debugging Challenges

Testing Across Service Boundaries

- Validating interactions is more complex than in monoliths
- Requires simulating real-world service dependencies

Advanced Testing Strategies Needed

- Contract, integration, and chaos testing are essential
- Ensures APIs and behaviors remain reliable

Debugging in a Distributed World

- Root cause analysis spans multiple services
- Requires distributed tracing and centralized logging





Network Latency and Performance



Service Calls Add Significant Latency

- Network requests are much slower than in-process calls
- Each service hop increases total response time



Cascading Latency Impacts User Experience

- Multiple dependent calls can slow down the entire system
- User-facing delays grow as services interact



Mitigation Strategies

- Implement caching to reduce repeated calls
- Use asynchronous communication and batch requests



Coordination, Security, and Summary



Coordination and Security Hurdles



Coordinating Service Changes

Synchronizing updates across multiple services is challenging.



API Versioning & Feature Rollouts

Careful planning needed to avoid breaking dependencies.



Expanded Security Risks

More endpoints increase attack surfaces and complexity.



Key Takeaways



Flexibility Comes with Complexity

Microservices offer agility but increase operational demands.



Success Demands New Skills and Tools

Teams need expertise in DevOps, automation, and distributed systems.



Careful Planning is Essential

Evaluate if microservices fit your team's needs and capabilities.